Rapport Projet
GPGPU

Groupe: GISTRE_3
Etudiant: valentin.delhommeau jon.gouspy ilan.mayeux alexandre.lacroix

Sommaire:

1. Introduction..... 3
2. Répartition des tâches par membre du groupe..... 3
3. Environnement, vidéos et métrique..... 4
3.1 Machine utilisé:.....4
3.2 Vidéo de test.....4
3.3 Métriques.....4
CUDA GPU Kernel Summary (cuda_gpu_kern_sum):.....4
CUDA API Summary (cuda_api_sum):..... 5
4. Versions..... 6
4.1 Baseline CPU..... 6
4.2 Portage GPU naïf (V1.1.0)..... 6
4.3 Changement du masque de float vers uint8_t (V1.1.1).....7
4.4 Copie du background sur changement (V1.1.2).....8
4.5 Hysteresis mémoire partager (V1.1.3)..... 10
4.6 Accès mémoire par le pitch (V1.1.4).....11
4.7 Ouverture séparable (V1.1.5).....12
4.8 Ouverture mémoire partagé (V1.1.6)..... 13
4.9 Évitement des itérations inutiles de l'hystérésis (V1.1.7)..... 14
4.10 Amélioration de l'organisation mémoire du BGE (V1.1.8).....15
5. Benchmark et graphique des performances des versions (CPU + GPU + GPU Optimisé).....17

1. Introduction

Le but du projet consiste à écrire un filtre GStreamer CUDA qui permet de séparer le fond et les objets mobiles dans un flux vidéo stabilisé. Puis, d'afficher les pixels en mouvement en rouge. C'est une étape préliminaire dans de nombreuses chaînes de traitement.

Pour notre projet, nous avons tout d'abord écrit une version CPU qui constitue notre référence pour comparer avec celle GPU. Puis nous avons fait une première version GPU naïf, qui nous sert de base avant de commencer nos optimisations. Nous l'avons optimisé en 8 étapes. Nous allons décrire et expliquer chacun de nos choix d'optimisation, ainsi que leur apport. Chaque étape est mesurée avec Nsight, qui nous permet de savoir où nous perdons du temps.

2. Répartition des tâches par membre du groupe

Membre	Tâches principales
Ilan Mayeux	Version C++ de référence (baseline) Portage direct (V1.1.0) Amélioration de l'organisation mémoire du BGE (v1.1.8) Benchmarks et Rapport
Jon Gouspy	Version C++ de référence (baseline) Portage direct (V1.1.0) Copie du background sur changement (V1.1.2) Ouverture séparable (V1.1.5) Ouverture mémoire partagé (V1.1.6) Benchmarks et Rapport
Valentin Delhommeau	Portage direct (V1.1.0) Changement float to uint8_t (V1.1.1) Copie du background sur changement (V1.1.2) Accès mémoire pitch (V1.1.4) Ouverture séparable (V1.1.5) Benchmarks et Rapport
Alexandre Lacroix	Version C++ de référence (baseline) Hysteresis mémoire partager (V1.1.3) Évitement des itérations inutiles de l'hystérésis (V1.1.7) Benchmarks et Rapport

3. Environnement, vidéos et métrique

Afin de réaliser un benchmark précis et d'analyser les différents points de ralentissement, toutes nos données ont été recueillies sur la même machine, vidéo et protocole. Cela nous permet de voir précisément les écarts provenant de l'implémentation et des algorithmes.

3.1 Machine utilisé:

Élément	Valeur
GPU	NVIDIA GeForce RTX 5050 Laptop GPU, 8151 MiB, 12.0
CPU	13th Gen Intel(R) Core(TM) i5-13500HX
CUDA / driver	NVIDIA-SMI 595.99.02 Driver Version: 595.99.02 CUDA Version: 13.2
Nsight Systems	NVIDIA Nsight Systems version 2025.6.3.343-256337165561v0
Compilateur et flags	Cuda compilation tools, release 13.2, V13.2.51

3.2 Vidéo de test

Vidéo	Résolution	Nombre de frames	Cadence
nuits_blanches.mp4	1920x1080	596	30 fps
3630-172488409_large.mp4	1920x1080	382	25 fps
ACET.mp4	776x1380	268	30 fps

3.3 Métriques

Pour chacune des versions nous allons nous appuyer sur ces différentes métriques grâce à **Nsight Systems**.

CUDA GPU Kernel Summary (*cuda_gpu_kern_sum*):

Métrique	Colonne	Ce qu'elle montre
Part de chaque kernel	Time (%)	Kernel dominant
Temps GPU du filtre	Total Time (ns)	Budget GPU
Lancements par frame	Instances	intérêt d'une fusion de kernels
Durée d'un appel de kernel	Med (ns) / Max (ns)	si Max très supérieur à Med alors les frames ponctuelles sont lentes

CUDA API Summary (*cuda_api_sum*):

Ici, nous allons principalement regarder cudaMemcpy, cudaMemcpy2D, cudaFree, cudaMemset, cudaMalloc.

Métrique	Colonne	Ce qu'elle montre
Part	Time (%)	API dominant
Temps	Total Time (ns)	Temps d'exécution de l'api
Nombre d'appel	Num Calls	Nombre d'appel

Pour calculer les FPS, nous mesurons la durée totale de traitement de la vidéo, puis nous divisons le nombre de frames par cette durée.

4. Versions

Tous les chiffres et comparaison dans cette section sont faites avec la vidéo: 3630-172488409_large.mp4

4.1 Baseline CPU

La version CPU implémente les mêmes étapes que la version GPU (src/process/cpu/), en C++ mono-thread, et sert de référence de correction pour toutes les versions GPU.

Mesure : Elle traite la vidéo de test à **1.7 FPS** (599.1 ms par frame).

Validation : Chaque version GPU est comparée visuellement à la sortie CPU. Les masques ne sont pas identiques au pixel près, car les tirages aléatoires de l'estimation du fond diffèrent entre CPU et GPU.

4.2 Portage GPU naïf (V1.1.0)

Un kernel par étape, un thread par pixel, mémoire globale uniquement, et aucune préoccupation de performance. À chaque frame :

- **Allocations :** MCProcess alloue ses buffers puis les libère (4 cudaMallocPitch pour la frame, le fond, le masque et un buffer temporaire ; 2 cudaMalloc pour l'hystérésis). BGEProcess alloue aussi sa propre copie de la frame quand il s'exécute.
- **Transferts :** À chaque frame, l'image et le fond sont envoyés séparément au GPU (2 × 6,2 Mo). Au retour, on rapatrie la frame traitée (6,2 Mo) et le masque complet stocké en float (8,3 Mo). De plus, à chaque mise à jour, le fond fait un aller-retour supplémentaire entre le CPU et le GPU.
- **Masque en float :** 4 octets par pixel pour une valeur qui tient dans [0f, 255f].
- **Ouverture :** Chaque pixel lit ses $(2r + 1)^2 = 81$ voisins en mémoire globale, pour l'érosion, puis pour la dilatation.
- **Hystérésis :** Le kernel d'hystérésis ne propage le marquage que d'un pixel par lancement : un objet de N pixels de long demande donc N itérations. Chacune enchaîne un cudaMemset du drapeau, un parcours complet de l'image, puis un cudaMemcpy synchrone qui bloque le CPU jusqu'à la fin du kernel. Le coût total est dominé par cette succession de synchronisations.

Le premier portage GPU sert de point de départ aux optimisations.

```
 Version v1.1.0 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  -------------------
    90,0    4 867 990 494     66 482      74 592,0      98 592  hysteresis_kernel
     3,0      186 950 855        382     489 347,0     498 627  morpho_kernel<MinOp>
     3,0      186 843 003        382     489 091,0     495 266  morpho_kernel<MaxOp>
     0,0       47 223 330         22   2 127 018,0   2 362 155  bge_process_kernel
     0,0       36 960 428        382     103 616,0     126 656  overlay_kernel
     0,0       36 471 951          1  36 471 951,0  36 471 951  init_curand_kernel
     0,0       30 471 005        382      81 616,0     115 488  mask_kernel
     0,0        9 288 296        382      24 320,0      24 641  binarize_kernel
     0,0        3 934 233        382      10 112,0      13 824  ConvertNV12BLtoNV12
     0,0           87 488         14       6 256,0       8 192  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ---------  ----------  ------------
    76,0    6 273 729 938     66 864   79 774,0   4 174 024  cudaMemcpy
     9,0      809 885 632      1 190  606 083,0  36 384 496  cudaMemcpy2D
     1,0      129 310 767      2 316   39 713,0   1 115 952  cudaFree
     1,0       94 106 766     66 864    1 235,0     358 584  cudaMemset
     0,0       15 968 561        764   22 252,0      79 081  cudaMalloc
```

Le masque est stocké en float alors que ses valeurs, des entiers de 0 à 255, tiennent sur un seul octet. Chaque accès au masque manipule donc 4 octets au lieu d'un. C'est le cas des kernels d'ouverture, d'hystérésis et de rendu, mais aussi de la copie du masque vers le CPU. Nous passons le masque en uint8_t, ce qui divise par 4 le volume de données lu, écrit et copié, sans changer le résultat.

4.3 Changement du masque de float vers uint8_t (V1.1.1)

Constat sur v1.1.0 : le masque, le buffer temporaire de l'ouverture et la copie de debug du masque vers le CPU utilise 4 octets par pixel pour des valeurs comprises entre 0 et 255.

Modification : tous les buffers et kernels du masque passent en uint8_t (mask_kernel, morpho_kernel, hysteresis_kernel, binarize_kernel, overlay_kernel, et les champs de MCProcess). Le rendu devient un calcul entier au lieu d'une division flottante.

Résultat : un lancement de l'ouverture passe de 489 à 441 µs (−10 %), et le rendu de 104 à 53 µs (−48 %). L'hystérésis (74 µs) et mask_kernel ne changent pas. Les copies passent de 2,59 à 1,95 ms par frame. Le temps GPU total passe de 16,9 à 15,8 ms (−6 %), et le FPS de 33,3 à 34,0 (+2 %)

Ce que montre v1.1.1 : L'hystérésis domine très largement, avec 90% du temps passé dans les kernels. Le profil révèle toutefois un autre surcoût, présent à chaque frame : 4 cudaMallocPitch, 2 cudaMalloc, 6 cudaFree et 3 cudaMemcpy2D par frame (1 188 appels, plus ceux du BGE). Ce problème étant simple à corriger, nous le traitons en premier et l'hystérésis fera l'objet de l'optimisation suivante.

```
 Version v1.1.1 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  -------------------
    90,0    4 788 123 502     65 828      73 760,0      98 912  hysteresis_kernel
     3,0      168 295 718        382     440 546,0     446 242  morpho_kernel<MinOp>
     3,0      168 218 310        382     440 354,0     445 826  morpho_kernel<MaxOp>
     0,0       45 870 683         21   2 125 386,0   3 198 703  bge_process_kernel
     0,0       36 672 979          1  36 672 979,0  36 672 979  init_curand_kernel
     0,0       30 648 057        382      78 368,0     112 928  mask_kernel
     0,0       20 427 363        382      53 408,0      65 281  overlay_kernel
     0,0        8 914 609        382      23 328,0      23 872  binarize_kernel
     0,0        4 182 426        382      10 688,0      13 633  ConvertNV12BLtoNV12
     0,0           87 808         14       6 224,0       8 320  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ---------  ----------  ------------
    75,0    5 893 132 948     65 828   79 485,0   1 967 645  cudaMemcpy
    10,0      795 499 484      1 188  587 125,0  36 547 343  cudaMemcpy2D
     1,0      116 955 214      2 315   33 895,0   1 132 468  cudaFree
     1,0       93 725 116     66 210    1 228,0     445 464  cudaMemset
     0,0       16 124 153        764   20 551,0     158 272  cudaMalloc
```

4.4 Copie du background sur changement (V1.1.2)

Constat sur v1.1.1 : 12 allocations ou libérations de mémoire par frame, et des allers-retours CPU ⇔ GPU pour la frame comme pour le fond.

Modification : nous avons réorganisé MCProcess et compute_cu :

- la frame est envoyée **une seule fois** sur le GPU (d_frame), puis partagée par BGEProcess et MCProcess ;
- le fond estimé **reste sur le GPU** (d_bg) : bge_process_kernel l'écrit directement, au lieu de le renvoyer au CPU puis de le recopier vers le GPU ;
- tous les buffers de travail (masque, masque temporaire, marqueurs, flag) sont **alloués une seule fois** dans le constructeur de MCProcess.

Résultat : l'objectif est atteint (plus aucune allocation par frame, une seule copie au lieu de deux, mais le gain global est faible : 15.8 → 15,2 ms (-4%), et les FPS passe de 34.0 à 35.8 (+5%).

```
 Version v1.1.2 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  -------------------
    90,0    4 801 813 823     65 847      73 729,0      98 976  hysteresis_kernel
     3,0      168 296 065        382     440 546,0     446 306  morpho_kernel<MinOp>
     3,0      168 233 730        382     440 386,0     446 274  morpho_kernel<MaxOp>
     0,0       44 707 732         20   2 128 410,0   3 273 551  bge_process_kernel
     0,0       36 671 373          1  36 671 373,0  36 671 373  init_curand_kernel
     0,0       28 954 433        382      74 336,0     228 641  mask_kernel
     0,0       20 194 663        382      52 448,0      62 304  overlay_kernel
     0,0        8 916 689        382      23 328,0      23 840  binarize_kernel
     0,0        4 824 849        382      12 576,0      15 649  ConvertNV12BLtoNV12
     0,0           87 936         14       6 336,0       8 288  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  -----------
    81,0    5 971 156 714     65 847   79 652,0  4 393 922  cudaMemcpy
     6,0      456 147 568        764  610 272,0  1 814 662  cudaMemcpy2D
     1,0       91 961 568     66 229    1 217,0    416 094  cudaMemset
     0,0        3 811 274          8  183 778,0  1 200 664  cudaFree
```

Analyse : les copies et allocations coûtaient environ 1,3 ms par frame, alors que l'hystérésis en coûte 12.7. Nsight le montrait déjà en **v1.1.1** : nous avons optimisé un poste secondaire avant le goulot principal.

4.5 Hysteresis mémoire partager (V1.1.3)

Constat sur v1.1.2 : l'hystérésis propage le marquage depuis les pixels « forts » (au-dessus du seuil haut) vers les pixels « moyens » (entre les deux seuils) qui leur sont connectés. Notre version ne propageait que d'un pixel par lancement ; chaque lancement relit et réécrit la mémoire globale. Soit 172 lancements par frame, avec chacun un cudaMemset et un cudaMemcpy.

Modification : chaque bloc charge une tuile dans la shared memory, puis propage à l'intérieur de la tuile jusqu'à stabilité avant d'écrire le résultat. Un lancement fait donc avancer la propagation sur toute la tuile, et non plus d'un seul pixel. Le kernel (hysteresis_block_kernel) se déroule en 3 étapes :

- **Chargement :** chaque thread charge son pixel (marqué s'il l'était déjà ou s'il dépasse le seuil haut). Les threads du bord chargent aussi le halo, c'est-à-dire les pixels voisins en bordure de tuile.
- **Propagation locale :** tant qu'au moins un thread a marqué un nouveau pixel, tout le bloc refait un tour. __syncthreads_or() synchronise le bloc et donne la même réponse à tous les threads, ce qui garantit qu'ils sortent de la boucle ensemble.
- **Écriture :** seuls les pixels nouvellement marqués sont écrits en mémoire globale.

Les frontières entre blocs se résolvent d'un lancement à l'autre : un marquage fait par un bloc devient visible à son voisin, via le halo, au lancement suivant. La boucle côté hôte continue jusqu'à ce qu'un lancement ne change plus rien.

Résultat : les lancements passent de **172 à 7.8 ms** par frame ($\frac{1}{22}$). Chaque lancement est plus long (74 → 111 µs, à cause de la boucle interne), mais l'hystérésis passe de 12,7 à **0,99 ms** par frame ($\frac{1}{13}$). Le temps GPU total passe de 15.2 à 3,2 ms ($\frac{1}{4.8}$), et le temps CPU bloqué dans cudaMemcpy de 15.6 à 2,3 ms par frame. L'hystérésis ne pèse plus que 27%. Les copies (1,24 ms, 39 %) et l'ouverture (0,88 ms, 28 %) deviennent visibles.

```
 Version v1.1.3 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  ----------------------
    45,0      380 571 721      2 986     111 136,0     307 714  hysteresis_block_kernel
    20,0      168 367 666        382     440 419,0     565 859  morpho_kernel<MinOp>
    20,0      168 177 841        382     440 227,0     449 283  morpho_kernel<MaxOp>
     4,0       36 652 694          1  36 652 694,0  36 652 694  init_curand_kernel
     3,0       31 352 143        382      80 385,0   1 116 327  mask_kernel
     2,0       20 755 116         10   2 070 190,0   2 364 304  bge_process_kernel
     2,0       19 823 489        382      51 744,0      55 296  overlay_kernel
     1,0        8 913 482        382      23 328,0      24 032  binarize_kernel
     0,0        4 932 558        382      12 864,0      16 064  ConvertNV12BLtoNV12
     0,0           88 288         14       6 336,0       8 256  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  ------------
    43,0      868 786 327      2 986  117 151,0  4 271 474  cudaMemcpy
    22,0      456 870 390        764  604 365,0  1 744 514  cudaMemcpy2D
     0,0        9 263 292      3 368    1 311,0     18 876  cudaMemset
     0,0        3 762 725          8  146 097,0  1 209 227  cudaFree
```

4.6 Accès mémoire par le pitch (V1.1.4)

Constat : les buffers du masque sont alloués avec cudaMallocPitch, mais les kernels les indexent en y * largeur + x, sans tenir compte du pitch.

Modification : ImageView gagne des accès getline(y) et at(x, y), utilisables sur CPU et sur GPU, qui passent par le pitch. Tous les kernels les utilisent, et binarize_kernel passe d'une grille 1D à une grille 2D.

Résultat : pas de gain. Le temps GPU passe de 3,20 à 3,26 ms (+2 %). L'ouverture ralentit légèrement (440 → 462 µs par lancement) et binarize_kernel double (23 → 49 µs) avec sa nouvelle grille 2D.

```
 Version v1.1.4 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  ----------------------
    43,0      373 929 517      2 894     112 577,0     286 562  hysteresis_block_kernel
    20,0      176 634 016        382     462 372,0     468 228  morpho_kernel<MinOp>
    20,0      176 554 440        382     462 179,0     468 451  morpho_kernel<MaxOp>
     4,0       36 600 630          1  36 600 630,0  36 600 630  init_curand_kernel
     3,0       31 542 329        382      80 784,0   1 123 944  mask_kernel
     2,0       20 721 035         10   2 063 837,0   2 368 272  bge_process_kernel
     2,0       20 137 935        382      52 576,0      59 520  overlay_kernel
     2,0       18 705 120        382      48 960,0      49 632  binarize_kernel
     0,0        4 913 409        382      12 800,0      16 000  ConvertNV12BLtoNV12
     0,0           87 424         14       6 224,0       8 192  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  -----------
    44,0      899 279 395      2 894  118 275,0  4 318 944  cudaMemcpy
    22,0      465 611 753        764  629 321,0    987 519  cudaMemcpy2D
     0,0        4 996 211      2 894    1 291,0     55 784  cudaMemset
     0,0        3 739 945          8  143 511,0  1 205 024  cudaFree
```

Analyse : en 1080p, une ligne du masque fait 1920 octets, soit 15 × 128 octets. Les lignes étaient donc déjà alignées sans utiliser le pitch, et cette version ne pouvait rien améliorer à cette résolution. Elle n'apporterait un gain que pour une largeur qui n'est pas multiple de 128 octets, où l'ancien indexage faisait commencer les lignes sur des adresses non alignées. Elle rend aussi le code cohérent : tous les buffers sont désormais indexés de la même façon, via le pitch.

Décision : conservée pour la lisibilité et la robustesse du code, pas pour la performance.

4.7 Ouverture séparable (V1.1.5)

Constat sur v1.1.4 : morpho_kernel lit $(2r+1)^2 = 81$ pixels par pixel, pour l'érosion comme pour la dilatation : 0,92 ms par frame.

Modification : le min (ou le max) sur une fenêtre carrée se décompose en un min sur la ligne, puis un min sur la colonne. morpho_kernel prend une direction (dx, dy) et fait une passe horizontale puis une passe verticale. Le résultat est identique, car la fenêtre est carrée.

Résultat : on passe de 81 à 2 × 9 = 18 lectures par pixel et par opération. Un lancement passe de 462 à 125 µs ($\frac{1}{3.7}$), mais il y en a 4 au lieu de 2 : l'ouverture passe de 0,92 à **0,50 ms par frame ($\frac{1}{1.8}$)**. Le temps GPU total passe de 3,26 à **2,86 ms (-12%)**.

```
 Version v1.1.5 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  ----------------------
    54,0      383 858 446      2 959     112 737,0     310 178  hysteresis_block_kernel
    13,0       96 414 123        764     124 833,0   1 153 543  morpho_kernel<MinOp>
    13,0       95 350 228        764     124 801,0     126 497  morpho_kernel<MaxOp>
     5,0       36 689 297          1  36 689 297,0  36 689 297  init_curand_kernel
     4,0       30 583 889        382      80 992,0     111 361  mask_kernel
     2,0       20 765 321         10   2 073 422,0   2 368 016  bge_process_kernel
     2,0       20 315 301        382      52 576,0     237 794  overlay_kernel
     2,0       18 698 727        382      48 929,0      49 569  binarize_kernel
     0,0        4 877 495        382      12 736,0      15 456  ConvertNV12BLtoNV12
     0,0           87 969         14       6 256,0       8 224  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  -----------
    38,0      727 576 876      2 959  118 442,0  3 673 426  cudaMemcpy
    25,0      475 740 314        764  630 010,0  1 833 474  cudaMemcpy2D
     0,0        5 427 692      2 959    1 303,0    268 437  cudaMemset
     0,0        3 779 799          8  167 299,0  1 185 454  cudaFree
```

Analyse : une passe de 125 µs est déjà proche du coût d'un simple parcours de l'image (mask_kernel : 81 µs, overlay_kernel : 53 µs). L'ouverture est désormais limitée par la mémoire plutôt que par le nombre de lectures.

4.8 Ouverture mémoire partagé (V1.1.6)

Constat sur v1.1.5 : l'ouverture demande 4 lancements par frame. La passe horizontale écrit une image intermédiaire complète en mémoire globale, que la passe verticale relit aussitôt. Chaque pixel est donc lu et écrit deux fois par opération, et les 9 voisins de chaque fenêtre passent par le cache L1/L2.

Modification : morpho_shared_kernel traite une opération (érosion ou dilatation) en un seul lancement. Chaque bloc de $B \times B = 32 \times 32$ threads :

- **Chargement** : il copie en mémoire partagée sa tuile et un halo de r pixels autour (in_tile, $T \times T$ avec $T = B + 2r = 40$). Les pixels hors de l'image reçoivent la valeur neutre (255 pour le min, 0 pour le max), ce qui donne le même résultat que de les ignorer.
- **Passe horizontale** : elle se fait entièrement en mémoire partagée, de in_tile vers h_tile (T lignes x B colonnes). Les lignes du halo sont conservées, car la passe verticale en a besoin.
- **Passe verticale** : elle lit h_tile, et chaque thread écrit son pixel une seule fois en mémoire globale.

La mémoire partagée est allouée dynamiquement au lancement : $T \times T + T \times B = 2880$ octets par bloc pour r = 4. Si le rayon dépasse la limite de 48 Ko par bloc (r > 87), on revient au kernel séparable de la v1.1.5. Le résultat est identique à celui de la v1.1.5.

Résultat : on passe de 4 à 2 lancements par frame. Un lancement prend 166 µs (érosion) et 181 µs (dilatation), contre 2 × 125 µs auparavant pour une opération. L'ouverture passe de 0,50 à 0.35 ms par frame (-32%). Le temps GPU total passe de 2,86 à 3,05 ms.

```
 Version v1.1.6 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  -------------------------
    68,0      569 857 243      4 103     113 857,0     393 827  hysteresis_block_kernel
     8,0       69 321 768        382     180 961,0     222 721  morpho_shared_kernel<MaxOp>
     7,0       63 241 732        382     165 537,0     167 970  morpho_shared_kernel<MinOp>
     4,0       36 710 418          1  36 710 418,0  36 710 418  init_curand_kernel
     3,0       31 792 531        382      80 544,0   1 118 312  mask_kernel
     2,0       20 748 777         10   2 071 101,0   2 362 384  bge_process_kernel
     2,0       20 263 078        382      52 544,0      59 264  overlay_kernel
     2,0       18 705 896        382      48 960,0      49 568  binarize_kernel
     0,0        4 899 363        382      12 800,0      15 712  ConvertNV12BLtoNV12
     0,0           87 937         14       6 320,0       8 353  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  ------------
    43,0      876 960 045      4 103  119 667,0  3 978 927  cudaMemcpy
    23,0      469 968 539        764  630 248,0  1 628 480  cudaMemcpy2D
     0,0        6 514 443      4 103    1 278,0     19 237  cudaMemset
     0,0        3 920 717          8  146 475,0  1 371 488  cudaFree
```

Analyse : le nombre d'accès à la mémoire globale par pixel et par opération est réduit. Les 18 lectures de voisins se font désormais en mémoire partagée au lieu du cache. Le gain est cependant limité par trois facteurs :

- le surcoût du halo, qui est relu par les blocs voisins
- les deux __syncthreads entre les étapes
- la taille des blocs : 1 024 threads par bloc, avec un GPU qui accepte 1 536 threads par SM. Un seul bloc tourne donc à la fois par SM, et ses attentes aux barrières ne sont pas recouvertes par un autre bloc.

4.9 Évitement des itérations inutiles de l'hystérésis (V1.1.7)

Constat sur v1.1.6 : l'hystérésis converge en environ 10.7 lancements par frame, mais chaque lancement relance **tous les blocs de l'image**, alors que la propagation ne progresse que le long de quelques fronts. Les autres blocs relisent leur tuile, ne trouvent rien à propager et réécrivent le même résultat. L'hystérésis coûte encore 1,22 ms par frame.

Modification : un bloc n'a du travail que si, au lancement précédent, **lui ou l'un de ses 4 voisins a modifié un pixel**. Sinon, sa tuile et son halo sont inchangés, donc son résultat aussi. Deux tableaux de drapeaux (un octet par bloc), alternés à chaque lancement, permettent de le savoir :

- **Saut** : en début de kernel, un bloc lit active_in pour lui-même et ses 4 voisins, et sort immédiatement si aucun n'est actif.

- **Marquage** : un bloc qui marque un nouveau pixel positionne active_out[b] = 1.
- **Échange** : côté hôte, les deux tableaux sont échangés entre deux lancements, et active_out est remis à zéro. Au premier lancement, tous les blocs sont actifs.

Le return anticipé précède toute barrière et ne dépend que de blockIdx : tous les threads d'un bloc sortent ensemble, sans risque de blocage. Le résultat est identique à la v1.1.6. Les tableaux sont alloués une seule fois dans le constructeur de MCProcess.

Résultat : le nombre de lancements ne change pas (≈ 10.7 par frame), car le suivi réduit le travail par lancement, pas le nombre de tours. Mais la durée moyenne des lancements passe de 114 µs à 53 µs. Pour un temps d'hysteresie par frame qui passe de 1,22 ms à 0,55 ms.

```
 Version v1.1.7 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)    Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  --------------------------
    59,0      386 268 207      4 023      52 640,0     416 675  hysteresis_block_kernel
    10,0       69 301 485        382     180 865,0     221 601  morpho_shared_kernel<MaxOp>
     9,0       64 269 999        382     165 569,0   1 189 800  morpho_shared_kernel<MinOp>
     5,0       36 589 041          1  36 589 041,0  36 589 041  init_curand_kernel
     4,0       31 987 545        382      80 865,0   1 124 455  mask_kernel
     3,0       20 735 880         10   2 063 549,0   2 372 271  bge_process_kernel
     3,0       20 279 847        382      52 545,0      63 232  overlay_kernel
     2,0       18 698 692        382      48 929,0      49 601  binarize_kernel
     0,0        4 870 210        382      12 672,0      16 064  ConvertNV12BLtoNV12
     0,0           87 616         14       6 256,0       8 288  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  ------------
    37,0      713 978 374      4 023   58 758,0  4 195 867  cudaMemcpy
    25,0      474 251 643        764  637 772,0  1 149 925  cudaMemcpy2D
     0,0       13 147 341      8 428    1 307,0     17 552  cudaMemset
     0,0        3 769 568         10   34 705,0  1 196 980  cudaFree
     0,0            9 329          2    4 664,0      5 785  cudaMalloc
```

Analyse : le gain sur le kernel est net, mais faible sur le temps total, car l'hystérésis ne domine plus. Les derniers tours restent autour de 50 µs : la grille complète est toujours lancée, et chaque tour garde ses cudaMemset et son cudaMemcpy synchrone.

4.10 Amélioration de l'organisation mémoire du BGE (V1.1.8)

Constat sur v1.1.7: Après toutes les optimisations précédentes,l'étape du Background Estimation Process prenait jusqu'à 3% de l'exécution. ~2.1 ms par appel. Le fonctionnement actuel est: un array de structure ReservoirState contenant weight/rgb pour chaque pixel associé.
Modification : En effectuant un réagencement de la mémoire de Array of Structure à Structure of Array, il est possible de rendre les accès mémoires beaucoup plus efficaces sans modification d'algorithme.

- Changement de la Reservoir et ReservoirState.
    - Avant:
        - using Reservoir = cuda::std::array<ReservoirState, RESERVOIR_K_MAX>
        - Image<Reservoir> d_rs_;
    - Après:
        - Image<uint32_t> d_weights;
          d_weights est un cube de dimension width x height x k avec k la taille du réservoir.
        - Image<uchar4> d_colors;
          d_colors est un cube de dimension width x height x k avec k la taille du réservoir.
        - La couleur rgb est stockée en uchar4 dans la mémoire pour l'alignement.
- Les accès mémoires sont désormais contiguës et la bandwidth réduite car moins de données à récupérées.:
    - Avant: Row y=0: [Pixel (0,0): (w0,c0)(w1,c1)...(wk,ck)] [Pixel (1,0): (w0,c0)(w1,c1)...(wk,ck)] …
    - Après: Rows 0 .. height-1 (Slot 0): [w(0,0)][w(1,0)][w(2,0)] ... 2*height-1 (Slot 1): [w(0,0)][w(1,0)][w(2,0)] ... …

Résultats: On observe une baisse du temps d'exécution à 1%, de 2.06ms à 1.10ms. Et ce, sans changer l'algorithme mais simplement l'ordonnancement mémoire.

```
 Version v1.1.8 : 3630-172488409_large.mp4 (382 frames)

 ** CUDA GPU Kernel Summary (cuda_gpu_kern_sum):

Time (%)  Total Time (ns)  Instances      Med (ns)     Max (ns)  Name
--------  ---------------  ---------  ------------  ----------  --------------------------
    60,0      386 389 542      4 020      52 992,0     416 899  hysteresis_block_kernel
    10,0       69 578 022        382     180 849,0     220 961  morpho_shared_kernel<MaxOp>
     9,0       63 247 584        382     165 569,0     168 002  morpho_shared_kernel<MinOp>
     5,0       36 589 459          1  36 589 459,0  36 589 459  init_curand_kernel
     4,0       30 989 696        382      81 248,0     103 936  mask_kernel
     3,0       20 207 547        382      52 544,0      59 681  overlay_kernel
     2,0       18 693 527        382      48 928,0      49 601  binarize_kernel
     1,0       11 294 857         10   1 096 951,0   1 441 801  bge_process_kernel
     0,0        4 893 086        382      12 736,0      15 456  ConvertNV12BLtoNV12
     0,0           87 937         14       6 256,0       8 256  MemsetD2D_BL


 ** CUDA API Summary (cuda_api_sum):

Time (%)  Total Time (ns)  Num Calls   Med (ns)   Max (ns)  Name
--------  ---------------  ---------  ---------  ---------  -----------
    36,0      687 540 231      4 020   58 900,0  2 911 258  cudaMemcpy
    25,0      475 094 219        764  636 337,0  1 716 931  cudaMemcpy2D
     0,0       12 805 804      8 422    1 297,0     26 451  cudaMemset
     0,0        1 048 153         11   39 003,0    289 035  cudaFree
     0,0            9 215          2    4 607,0      5 202  cudaMalloc
```

Analyse: L'ordonnancement de mémoire est bien plus adapté au problème ce qui permet un gain significatif de performance. Le prochain problème pour des gains de performance supplémentaire consistera à améliorer la bande passante sur la mémoire globale puisqu'il y a presque 3 lectures de d_weights et de d_colors par pixel.

5. Benchmark et graphique des performances des versions (CPU + GPU + GPU Optimisé)

Performances sur “nuits_blanches.mp4” (1920x1080), Speedup relatif au CPU

Version	FPS	ms/frame	Speedup
CPU (baseline)	2.5	399.3	1.00x
GPU v1.1.0 (base)	33,5	29,8	13,4x
GPU v1.1.1	35,1	28,5	14,0x
GPU v1.1.2	36,8	27,2	14,7
GPU v1.1.3	71,2	14,1	28,3x
GPU v1.1.4	70,9	14,1	28,3x
GPU v1.1.5	73,7	13,6	29,4x
GPU v1.1.6	72,9	13,7	29,1x
GPU v1.1.7	75,6	13,2	30,2x
GPU v1.1.8	76,0	13,2	30,4

Performance sur “3630-172488409_large.mp4” (1920x1080), Speedup relatif au CPU

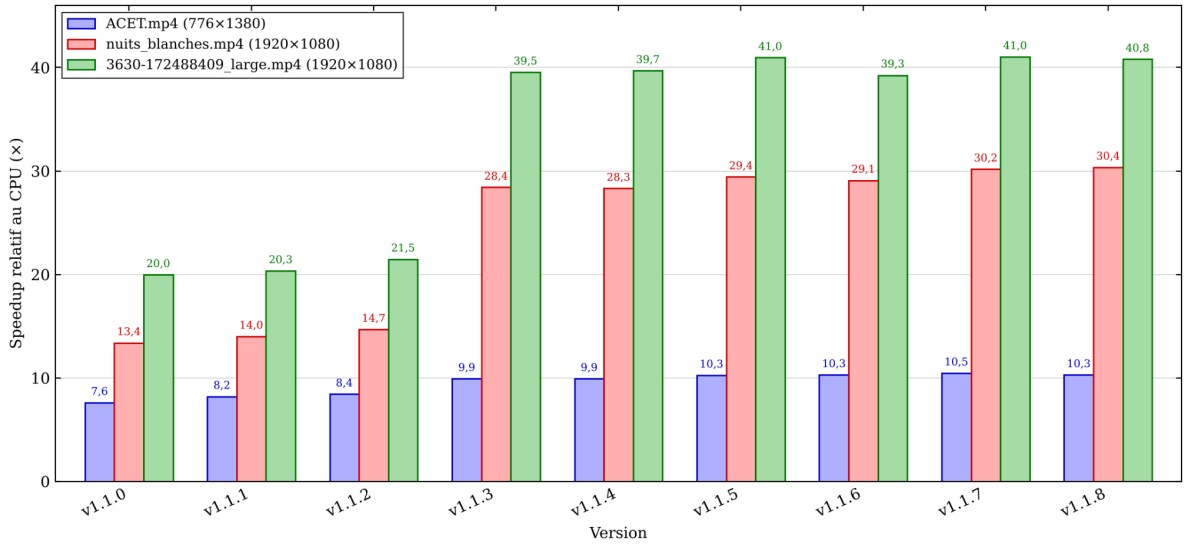
Version	FPS	ms/frame	Speedup
CPU (baseline)	1.7	599.1	1x
GPU v1.1.0 (base)	33,3	30,0	20,0x
GPU v1.1.1	34,0	29,5	20,3x
GPU v1.1.2	35,8	27,9	21,5x
GPU v1.1.3	66,0	15,2	39,7x
GPU v1.1.4	66,2	15,1	39,7x
GPU v1.1.5	68,4	14,6	41,0x
GPU v1.1.6	65,5	15,3	39,3x
GPU v1.1.7	68,5	14,6	41,0x
GPU v1.1.8	68,1	14,7	40,8x

Performance sur “ACET.mp4” (776x1380), Speedup relatif au CPU

Version	FPS	ms/frame	Speedup
CPU (baseline)	9.5	105.5	1x
GPU v1.1.0 (base)	72,1	13,9	7,6x
GPU v1.1.1	77,8	12,9	8,2x
GPU v1.1.2	79,9	12,5	8,4x
GPU v1.1.3	94,1	10,6	9,9x
GPU v1.1.4	94,0	10,6	9,9x
GPU v1.1.5	97,2	10,3	10,3x
GPU v1.1.6	97,5	10,3	10,3x
GPU v1.1.7	99,2	10,1	10,5x
GPU v1.1.8	97,5	10,3	10,3x

Table de comparaison des 3 vidéo, Speedup relatif au CPU

Version	ACET		nuits_blanches		3630-172488409_large	
	FPS	Speedup	FPS	Speedup	FPS	Speedup
CPU (baseline)	9.5	1x	2.5	1.00x	1.7	1x
GPU v1.1.0 (base)	72,1	7,6x	33,5	13,4x	33,3	20,0x
GPU v1.1.1	77,8	8,2x	35,1	14,0x	34,0	20,3x
GPU v1.1.2	79,9	8,4x	36,8	14,7x	35,8	21,5x
GPU v1.1.3	94,1	9,9x	71,2	28,3x	66,0	39,7x
GPU v1.1.4	94,0	9,9x	70,9	28,3x	66,2	39,7x
GPU v1.1.5	97,2	10,3x	73,7	29,4x	68,4	41,0x
GPU v1.1.6	97,5	10,3x	72,9	29,1x	65,5	39,3x
GPU v1.1.7	99,2	10,5x	75,6	30,2x	68,5	41,0x
GPU v1.1.8	97,5	10,3x	76,0	30,4x	68,1	40,8x



Les benchmarks FPS du traitement des vidéos ont été réalisés avec les mêmes paramètres (--low=10 --high=50) ainsi que la même configuration pour toutes les vidéos et les versions.

Tout d'abord, on observe que le speedup obtenu avec le GPU augmente significativement avec la résolution de la vidéo. En effet, le meilleur speedup passe d'environ 10,5 pour ACET (776×1380, soit 1,1 million de pixels) à environ 30,4 pour nuits_blanches et 41,0 pour 3630 (1920×1080, soit 2,1 millions de pixels). Pour un nombre de pixels à peu près doublé, le speedup est donc multiplié par 3 à 4. On note aussi qu'à partir de la version v1.1.3, le FPS plafonne : autour de 95 à 99 FPS pour ACET, et de 66 à 76 FPS en 1080p.

Les optimisations de v1.1.4 à v1.1.8 réduisent pourtant encore le temps GPU, mais leurs écarts de FPS (1 à 4 %) sont du même ordre que la variation entre deux exécutions.

Cette évolution s'explique par le fait qu'à partir de v1.1.3, les kernels s'exécutent très rapidement. En 1080p, le calcul GPU ne prend plus qu'environ 2,4 ms sur les 14,6 ms d'une frame. Le temps d'exécution total est alors majoritairement dominé par ce qui ne dépend pas de nos kernels : le décodage de la vidéo, GStreamer, les transferts entre le CPU et le GPU et le coût de l'API CUDA. À faible résolution, la part de ce surcoût fixe est encore plus grande, ce qui limite le speedup. À haute résolution, la quantité de calcul augmente, et le GPU peut mieux exploiter son parallélisme face au CPU.

Le contenu de la vidéo joue aussi un rôle. À résolution égale, le CPU est plus lent sur 3630 (1,7 FPS) que sur nuits_blanches (2,5 FPS) : la foule dense produit un masque très chargé, et l'hystérésis a beaucoup plus de pixels à propager. Le GPU est presque insensible à cette différence (68 contre 76 FPS), d'où un speedup plus élevé sur 3630.

Enfin, on observe que le plus grand gain provient de l'hystérésis par tuiles en mémoire partagée (v1.1.2 à v1.1.3). Entre ces deux versions, le FPS augmente de 93 % sur nuits_blanches et de 84 % sur 3630, mais de seulement 18 % sur ACET. Cette différence vient, là encore, de la nature de la vidéo : sur ACET, petite et avec peu de mouvement, l'hystérésis converge vite et pèse peu dans le temps total. Il y a donc moins à gagner en l'accélérant.